

STUDENT MARKS & ATTENDANCE MANAGEMENT SYSTEM

Python Mini Project Report

Submitted By:

Student Name: DHEERAJ KUSHWAH

Registration No.: 26MIM10099

Institution:

VIT Bhopal University

Academic Year: 2026–27

1. Introduction

The Student Marks & Attendance Management System is a menu-driven Python application designed to manage basic student academic records. It allows the user to add, search, update and delete student information, enter marks and attendance, calculate results, and save records in a CSV file.

The project demonstrates fundamental Python programming concepts such as functions, lists, dictionaries, loops, conditional statements, file handling, the csv module, the os module, and date/time handling.

2. Problem Statement

Managing student marks and attendance manually can be time-consuming and may lead to calculation or record-keeping errors. A simple computerized system is required to store student details, maintain subject marks and attendance, calculate percentage and grade, and save the information for later use.

The proposed system provides a simple command-line solution for these requirements.

3. Functional Requirements

- Add Student: Enter roll number, name and course.
- Search Student: Find a student using the roll number.
- Update Student: Modify name and course details.
- Delete Student: Remove a student record.
- Enter Marks and Attendance: Store marks for Python, Maths and English along with classes held and attended.

- Show Result: Calculate total marks, percentage, grade and attendance eligibility.
- Save Data: Store records in students.csv.
- Check CSV File: Verify whether the CSV file exists.
- Exit: Close the application.

4. Non-functional Requirements

- Usability: The menu and prompts should be simple and easy to understand.
- Performance: Operations should complete quickly for a normal classroom-sized dataset.
- Reliability: Calculations should follow the defined grading and attendance rules.
- Maintainability: Functions are separated according to their tasks, making the code easier to modify.
- Portability: The program can run on systems with Python installed.
- Data persistence: Student records can be exported to CSV for storage outside the running program.

5. System Architecture

The system follows a simple modular architecture. The main menu accepts the user's choice and calls the appropriate function. Student records are maintained in an in-memory list of dictionaries. The result module performs percentage, grade and attendance calculations. The file-management module writes the records to a CSV file.

User	Main Menu	Student Management	Marks & Attendance	Result Module	CSV Storage
Input	Choice	Add/Search/Update/Delete	Enter marks & Attendance	Calculate	Save/Check

6. Design Diagrams

6.1 Use Case Diagram

Actor	Use Case
User	Add Student
User	Search Student
User	Update Student
User	Delete Student
User	Enter Marks & Attendance
User	Show Result
User	Save Data / Check CSV

6.2 Workflow Diagram

Start → Display Main Menu → Select Operation → Perform Student/Marks Operation → Return to Menu → Save Data if required → Exit → End

6.3 Sequence Diagram

User → Main Menu: Select option
Main Menu → Function: Call selected function
Function → Students List: Read/modify record
Function → User: Display result/message
User → Save Function: Request CSV save
Save Function → students.csv: Write records

6.4 Class/Component Diagram

Component	Responsibility
Main Menu	Controls program flow and user choices
Student Management	Add, search, update and delete records
Marks & Attendance	Enter academic data
Result Module	Calculate total, percentage, grade and attendance status
File Management	Save records and check CSV availability

6.5 ER Diagram (Storage Used)

Entity: STUDENT	Attributes
Primary Key	roll
Student Details	name, course
Marks	python, maths, english
Attendance	held, attendance

7. Design Decisions & Rationale

Python was selected because it is easy to learn and provides built-in modules suitable for this project.

A list of dictionaries was used because it provides a simple structure for storing multiple student records without requiring a database server.

CSV was selected as the storage format because it is lightweight, human-readable and can be opened using spreadsheet software.

Functions were used for each major operation to improve readability, reuse and maintainability.

The grading system uses percentage ranges: A+ for 90 or above, A for 80–89.99, B for 70–79.99, C for 60–69.99, D for 50–59.99, and F below 50.

Attendance eligibility is set to eligible when calculated attendance is 75% or above; otherwise the status is shortage.

8. Implementation Details

The program imports csv for CSV file operations, os for checking file existence, and datetime for adding the current date to saved records.

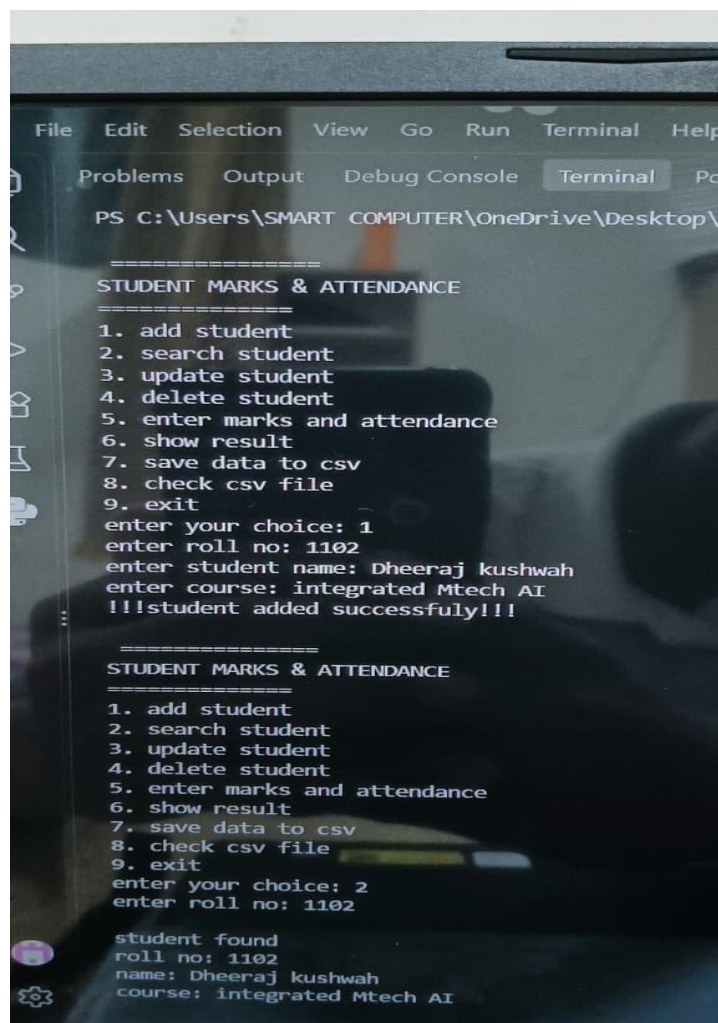
Student records are stored as dictionaries containing roll, name, course, subject marks and attendance fields. The main while loop repeatedly displays the menu until the user chooses Exit.

The calculate_result() function adds the three subject marks, calculates percentage by dividing the total by three, determines the grade using conditional statements, and calculates attendance using attended classes divided by classes held.

9. Screenshots / Results

The expected output includes a main menu with nine options. After adding a student, the program displays a success message. Search displays the student's roll number, name and course. The result screen displays total, percentage, grade, attendance percentage and attendance status.

For the final project submission, screenshots of the actual program execution can be inserted in this section, such as: Main Menu, Add Student, Enter Marks & Attendance, Show Result, and students.csv.



```
File Edit Selection View Go Run Terminal Help
Problems Output Debug Console Terminal Po
PS C:\Users\SMART COMPUTER\OneDrive\Desktop\j

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 1
enter roll no: 1102
enter student name: Dheeraj kushwah
enter course: Integrated Mtech AI
!!!student added successfully!!!

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 2
enter roll no: 1102

student found
roll no: 1102
name: Dheeraj kushwah
course: Integrated Mtech AI
```

```
File Edit Selection View Go Run Terminal
Problems Output Debug Console Terminal

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 5
enter roll no: 1102

student found
roll no: 1102
name: Dheeraj kushwah
course: integrated Mtech AI
python marks: 88
maths marks: 99
english marks: 90
class held: 90
class attendance: 100
marks and data saved..

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 6
```

```
File Edit Selection View Go Run Terminal Help
Problems Output Debug Console Terminal Ports

9. exit
enter your choice: 6
enter roll no: 1102

student found
roll no: 1102
name: Dheeraj kushwah
course: integrated Mtech AI

=====RESULT=====
name: Dheeraj kushwah
total: 277.0
percentage: 92.33
grade: A+
attendance: 111.11 %
attendance student: eligible

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 7
data saved in students.csv

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
```

```
File Edit Selection View Go Run Terminal
Problems Output Debug Console Terminal

3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 7
data saved in students.csv

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 8
csv file is available..

=====
STUDENT MARKS & ATTENDANCE
=====
1. add student
2. search student
3. update student
4. delete student
5. enter marks and attendance
6. show result
7. save data to csv
8. check csv file
9. exit
enter your choice: 9
thankyou!!
```

10. Testing Approach

Testing was performed by checking each menu option separately and verifying the displayed output.

Test cases include: adding a new student, searching an existing and non-existing roll number, updating a student, deleting a student, entering valid marks, calculating different grades, checking attendance at and below 75%, saving the CSV file, and checking whether the CSV file exists.

Test Case	Input/Condition	Expected Result
Add	Valid student details	Student added
Search	Existing roll	Student details displayed
Search	Unknown roll	Student not found
Result	Average = 90+	Grade A+
Attendance	Attendance >= 75%	Eligible
Attendance	Attendance < 75%	Shortage
Save	Save option	students.csv created/updated

10. Challenges Faced

- Managing multiple student records using a list of dictionaries.
- Searching and updating the correct record using the roll number.
- Correctly calculating percentage and grade using conditional statements.
- Handling attendance calculation when the number of classes held is zero.
- Writing structured data into a CSV file with appropriate headers.
- Maintaining the program flow using a continuous menu loop.
- Preventing common input and data-entry errors can be improved further through validation.

11. Conclusion

The Student Marks & Attendance Management System successfully demonstrates how Python can be used to solve a basic academic record-management problem. The project combines student management, result calculation, attendance calculation and CSV file handling in one menu-driven application.

The system can be extended in the future by adding password-based login, input validation, automatic CSV loading, database connectivity, multiple subjects, report generation and a graphical user interface.